Redistributing this file (including partially) to CourseHero or other public websites is strictly prohibited.

COMP5434 Project

1. Overview

There are two datasets (Dataset 1, and Dataset 2) in this project. For each dataset, you need to develop a solution to predict the class labels of the testing records in the dataset. A record in the datasets contains several attributes. The real-world meaning of the class labels and attributes are removed, which will let you sense the power of big data analytics to generate accurate results *without* expert knowledge of a certain application domain.

2. Submission Deadline

a. Predictions on Kaggle: **11:55 PM, November 14th, 2026**
b. Presentation slides, report and code: **08:00 AM, November 16th, 2026**

3. Data Description and Instructions

For each dataset, we provide its training data (train.csv), testing data (test.csv), and submission sample (sample_submission.csv).

Dataset 1

Overview: This dataset records information about the weather for some days. Your goal is to train a model to predict **the weather** based on the training set.

Training data: *train.csv*

- Each row is a data record with its id, 10 attributes, and a label.
- The last cell of each row ("*label*" column) is "*class label"* in integer, which is our classification target.
    - This label indicates the weather (1, 2, 3, and 4), 1 for *Sunny*, 2 for *Cloudy*, 3 for *Rainy*, and 4 for *Snowy*.
- Training data includes training records with ground-truth class labels.
- Use the training data to train your solution.

Testing data: *test.csv*

- Each row is a data record with its id and 10 attributes. The attributes are the same as those of *train.csv*, but the *label* is missing.
- Use your method to get your predicted labels of the testing records in *test.csv* and generate submission file which includes your predictions in the format of *sample_submission.csv*.
- We will obtain your solutions' performance based on your predicted labels for testing data.
- Our evaluation is based on the **Macro-F1** metric.
    - A detailed explanation of the calculation of the metrics:
        - https://scikit-learn.org/stable/modules/model_evaluation.html#multiclass-and-multilabel-classification
    - Example: A third-party library to calculate the Macro-F1 scores:
        - https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html

- After generating your predicted labels, you need to submit it to our online platform.

Sample Submission: *sample_submission.csv*
- This file is a sample of your submission about the predictions of *test.csv*.
- This file includes two columns.
    - *id*: the identifier of each record in *test.csv*.
    - *label*: the prediction of this incident.

Dataset 2
Overview: Your goal is to train a model to predict **the rating of each review**.

Training data: *train.csv*
- Each row corresponds to a single review and contains the following fields: *id* (the id of the review), *title* (the review title), *comment* (the review text), *user_id* (ID of the user who posted the review), *prod_id* (ID of the reviewed product), *parent_prod_id* (Parent ID of the product. For example, same products with different colors share identical parent ID), *time* (timestamp when the review was posted), *votes* (number of users who found the review helpful), *purchased* (whether the user purchased the product), and ground-truth *rating* (an integer from 1 to 5).
- The last cell of each row ("*rating*" column) is the prediction target
- Training data includes training records with ground truth.

Testing data: *test.csv*
- Each row is a review. The attributes are the same as those of *train.csv*, but the *rating* column is missing.
- Use your method to get your predicted ratings of the testing records in *test.csv* and generate submission file which includes your predictions in the format of *sample_submission.csv*.
- We will obtain your solutions' performance based on your prediction for testing data.

Sample Submission: *sample_submission.csv*
- This file is a sample of your submission about the predictions of *test.csv*.
- This file includes two columns.
    - *id*: the identifier of each review record in *test.csv*.
    - *rating*: the predicted rating of this review.

Additional Requirements for Dataset 2
- Develop a solution to predict review ratings that is both accurate and efficient. The solution **must** incorporate at least one big data parallel/distributed processing framework (e.g., Hadoop, Spark, Flink, Storm, or any other framework that you think is necessary). You may use any algorithms or techniques, *whether covered in the course or not*
- Any programming language is permitted. Your code must be clean, well-documented (with sufficient comments), and accompanied by a high-quality README providing step-by-step execution instructions for reproducibility. Your code will be run to verify the reported efficiency metrics required below.

- Effectiveness evaluation is based on the **RMSE (root mean squared error)** metric.
- Some Hints:
    - Approach the problem as either a regression or classification task.
    - Consider integrating natural language processing (for text fields like title and comment), graph-based methods (to model user–product relationships), or a combination of both.
    - Your solution must incorporate at least one big data parallel/distributed processing framework, whether covered in the course or not.
    - Resource Options:
        - Use a single machine configured in pseudo-distributed mode to simulate parallel processing.
        - Alternatively, use the departmental HPC platform: https://hpc.comp.polyu.edu.hk/howto/
        - Any cloud services you think necessary
    - **Mandatory Report Specifications**: In your report, please report
        - **Hardware details:** CPU model, number of servers/threads, GPU model (if applicable).
        - **Efficiency evaluation**: Training time per epoch, Total offline time (preprocessing + training), Total inference time for all test records

4. Project Platform

We use Kaggle (https://kaggle.com) to evaluate your predictions.
We hold two competitions for these two tasks. You should attend them as a team. **Your team name in Kaggle must be the same as the name in our team list**, which has been released in Canvas.
You may refer to https://www.kaggle.com/docs/competitions and the video (Kaggle-instruction) to know how to use Kaggle.

Competition Links:

1. Task 1: https://www.kaggle.com/t/d6784c43253c4aaf8659a4a6eb17c697
2. Task 2: https://www.kaggle.com/t/b818816f6d244be7afc696c87bf48fbf

Important Notes:

1. When you submit your prediction file, you can check your Macro-F1 score and rank in all teams immediately.
    a. However, it is a TEMPORARY performance because it is calculated from A PART OF (about 50%) testing data (called Public Leaderboard).
    b. When the competition ends, it will evaluate your submission from THE OTHER PART OF (another about 50%) testing data and update your ranks (called Private Leaderboard).
    c. This performance in Private Leaderboard will be used to grade your methods.
    d. **You should select two submissions for each competition BEFORE THE COMPETITION END.** Your final score and placement at the end of the competition will be whichever selected submission performed best on the private leaderboard. If you do not select submission(s) to be scored before the competition closes, the platform will automatically select those

which performed the highest on the public leaderboard.

2. The competitions will end at **11:55 PM, November 14th, 2026**
3. The finalized rank will be released at **08:00 AM, November 15th, 2026**
4. You can only submit **10 times per day** to check your temporary performance, due to the resource limitation.

5. Tasks and Requirements

- Develop two solutions, one for each dataset, to predict the class label of each data record.
- You can develop any solutions, based on *either the algorithms introduced in this subject or the methods beyond the course content.*
- Any programming language
    - Your code should be clean and well-documented (e.g., with sufficient comments)
- You can use low-level third-party packages to facilitate your implementation.
- **Your implementation should involve sufficient technical details developed by yourselves.**
    - DO NOT simply call ready-to-use classification models provided in existing packages, as a **Blackbox**, to finish the project.
        - For example, Spark provides an implementation of Naïve Bayes classifier (https://spark.apache.org/docs/latest/mllib-naive-bayes.html), which, after preparing the data, can be used to train the data with just a simple function call, as shown in the following (Cases like this example will lead to point deduction)

```scala
import org.apache.spark.mllib.classification.{NaiveBayes, NaiveBayesModel}
import org.apache.spark.mllib.util.MLUtils

// Load and parse the data file.
val data = MLUtils.loadLibSVMFile(sc, "data/mllib/sample_libsvm_data.txt")

// Split data into training (60%) and test (40%).
val Array(training, test) = data.randomSplit(Array(0.6, 0.4))
      Calling a read-to-use classifier as a blackbox leads to point deduction
val model = NaiveBayes.train(training, lambda = 1.0, modelType = "multinomial")

val predictionAndLabel = test.map(p => (model.predict(p.features), p.label))
val accuracy = 1.0 * predictionAndLabel.filter(x => x._1 == x._2).count() / test.count()

// Save and load model
model.save(sc, "target/tmp/myNaiveBayesModel")
val sameModel = NaiveBayesModel.load(sc, "target/tmp/myNaiveBayesModel")
```

- **You are ALLOWED to use third-parity libraries**. As long as your implementation involves reasonable algorithmic details for solving the problem, then it is fine. Unless it is too obvious, we will be very moderate when deciding if an implementation is solely based on Blackbox algorithms.
    - **i.e., let us see your effort, instead of purely calling black box algorithms.**

6. Hints

- *Note that data may be dirty.*
- Before running your solutions, you may need to follow the 5-step data analytical process (See Lecture 1 Slides) to prepare the data, in order to achieve higher performance.
- Try your best to improve your solutions' performance.

7. Report File

- Include your team name, your names, your student ids in the report
- *Suggested report structure:*
    - Problem definition
    - Data Analysis and Model design
    - Solution and implementation details
        - Probably follow the major steps in the 5-step data analytical process
        - Your unique technical designs to improve effectiveness and efficiency.
        - Your solution architecture and technical details of each component
        - Any other content that highlights your own designs
    - Performance evaluation and discussion
        - Report Macro-F1 (for Dataset 1) and RMSE (for Dataset 2) and any additional evaluation metrics considered relevant.
        - Include hardware specifications: CPU model, number of servers/threads, GPU model (if used).
        - Report timing metrics: training time per epoch, total offline time (preprocessing + training), and total inference time for all test records.
        - Analyze experimental results, including ablation studies or comparative evaluations.
        - Draw meaningful conclusions from the findings.
    - Summary of discoveries and future work.
    - **A contribution table**
        - In your report, include a contribution table indicating your percentage of contributions, in total 100% (grades will be adjusted accordingly).
        - Any teams without a contribution table will not be graded.
    - References

8. Presentation Guidelines

- 5-10 slide explaining your core ideas in the report
- **Face2Face presentation**
- Each team will have about 7 minutes for a presentation and 3 minutes for Q&A session [depending on the number of teams].
    - Do not exceed the time requirement.

9. Submission Guidelines:

- Create folder with name *TeamName*.
    - *TeamName* is the name in the team list on Canvas.
    - Put all your code together in a folder with name *code*.

    - Rename your slides with name *slides* (with the extension name, such as *pptx* and *pdf*).
    - Rename your report with name *report* (with the extension name, such as *pdf*).
    - Put *code*, *slides* and *report* in the folder *TeamName*.
    - You should replace *TeamName* to your actual team's name.
- Compress this folder as one zip file.
    - **Your submitted predictions in Kaggle must be able to be generated by using your codes. Otherwise, the marks of corresponding part will be 0.**
        - There are many ways to keep your code reproducible, i.e., we can get the same results in every running. For example, this webpage (https://pytorch.org/docs/stable/notes/randomness.html) includes some methods to keep it reproducible in Python and PyTorch (a machine learning library). Another webpage (https://keras.io/examples/keras_recipes/reproducibility_recipes/) introduces such methods for Keras (another machine learning library). For other libraries you plan to use in your code, please refer to its corresponding libraries to learn how to keep it reproducible.
- Follow the below example to name your zip file by replacing "*TeamName*" to your actual team's name:
    - *TeamName.zip*
- Your submission should be submitted by your **TEAM COORDINATOR** and the deadline is **08:00 AM, November 16th, 2026**.

10. Grading Rubrics

- Code (6%):
    - Code quality
        - Technical designs for efficiency and accuracy
    - Code performance on testing data
- Report (5%)
- Presentation (4%)

Outcome Presentations	%	**A+/A/A-**	**B+/B/B-**	**C+/C/C-**	**D+/D**	**F**
Code quality	3%	Programs are well-organized, makes good use of whitespace and comments. Variables have helpful names.	Programs are well-organized, easy to read and understand.	Programs can be read and is in a logical order.	Programs are runnable but barely readable.	Absent
Report Content	5%	Excellent, comprehensive and in-depth analysis with concrete facts/evidence	Clear analysis with good analysis supported by plenty of facts/evidence	Basic analysis with some level of facts/evidence	Barely relevant analysis with minimal facts/evidence	Absent
Presentation quality	4%	Very clear and logical	Good, easy to follow	Understandable, structured	Barely understandable	Absent